

Apuntes Completos de Java: Guía de Referencia Esencial V2

Esta es la guía definitiva de referencia rápida con explicaciones conceptuales avanzadas y ejemplos de código detallados para cada uno de los componentes fundamentales de Java. Diseñada para asimilar conceptos clave de forma directa y práctica.

Bloque 1: Tipos de Datos Básicos

1.1 Tipo int

Almacena números enteros de 32 bits, positivos o negativos, sin decimales.

```
int edad = 25;
int usuariosActivos = 45;
usuariosActivos++; // Incrementa el valor en 1 (ahora vale 46)
System.out.println("Usuarios: " + usuariosActivos);
```

1.2 Tipo double

Almacena números con punto decimal de doble precisión (64 bits), ideal para cálculos matemáticos precisos, dinero o porcentajes.

```
double precio = 99.99;
double descuento = 0.15;
double total = precio * (1.0 - descuento);
System.out.println("Total con descuento: " + total);
```

1.3 Tipo boolean

Admite únicamente dos valores lógicos booleanos: true o false.

```
boolean tieneAcceso = true;
boolean juegoTerminado = false;
if (tieneAcceso) {
    System.out.println("El jugador puede entrar.");
}
```

1.4 Tipo String

Representa cadenas de texto estructuradas. Al ser una clase nativa de Java, cuenta con métodos propios y se declara con mayúscula.

```
String nombre = "Aragorn";  
String saludo = "Hola, " + nombre + "!";  
System.out.println(saludo);
```

Bloque 2: Control de Flujo y Entrada de Datos

2.1 Condicionales (if-else)

Evalúa una expresión booleana para bifurcar el camino de ejecución del programa.

```
int vida = 50;  
if (vida > 70) {  
    System.out.println("Estado: Excelente");  
} else if (vida >= 20) {  
    System.out.println("Estado: Alerta");  
} else {  
    System.out.println("Estado: Crítico");  
}
```

2.2 Bucle for

Estructura de repetición controlada por un contador interno, idónea cuando conocemos el número exacto de iteraciones.

```
for (int i = 1; i <= 3; i++) {  
    System.out.println("Vuelta número: " + i);  
}
```

2.3 Bucle while

Repite un bloque de instrucciones mientras una condición lógica externa siga siendo verdadera.

```
int flechas = 3;  
while (flechas > 0) {  
    System.out.println("¡Disparo! Flechas restantes: " + flechas);  
    flechas--;  
}
```

2.4 Bucle do-while

Garantiza de forma estricta la ejecución del bloque interno **al menos una vez**, evaluando la condición de salida al final de la vuelta.

```
int energia = 0;
do {
    System.out.println("Acción obligatoria: El personaje intenta ponerse en pie.");
} while (energia > 10);
```

2.5 Entrada de datos con Scanner

Clase de la librería estándar utilizada para parsear y leer la entrada del usuario por consola.

```
import java.util.Scanner;

public class EjemploScanner {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("Introduce el nombre de tu guerrero: ");
        String nombreHroe = teclado.nextLine();
        System.out.println("Héroe registrado: " + nombreHroe);
    }
}
```

Bloque 3: Estructura del Código y Métodos

3.1 Métodos con parámetros y retorno

Bloques modulares de código aislados diseñados para recibir variables de entrada, procesar información y retornar un valor explícito.

```
public class CalculadorCombate {
    public static int calcularDanioFinal(int ataqueBase, int defensaEnemigo) {
        int danio = ataqueBase - defensaEnemigo;
        if (danio < 0) {
            return 0;
        }
        return danio;
    }
}
```

Bloque 4: Programación Orientada a Objetos (POO)



4.1 Clases, Objetos y Constructores

La clase representa el molde genérico estructurado, mientras que el objeto es la instancia materializada e independiente en la memoria dinámica.

```
public class Item {  
    private String nombre;  
    private int peso;  
  
    // Constructor para inicializar el objeto  
    public Item(String nombreInicial, int pesoInicial) {  
        nombre = nombreInicial;  
        peso = pesoInicial;  
    }  
  
    public void usar() {  
        System.out.println("Usando el objeto: " + nombre);  
    }  
}
```

4.2 Variables Estáticas (static)

Variables globales vinculadas de forma directa a la estructura de la clase y no a las instancias individuales; su valor es compartido unificadamente.

```
public class Enemigo {  
    public String nombre;  
    // Atributo estático compartido globalmente  
    public static int totalEnemigosMundo = 0;  
  
    public Enemigo(String nombre) {  
        this.nombre = nombre;  
        totalEnemigosMundo++; // Incrementa para todas las instancias simultáneamente  
    }  
}
```

Bloque 5: POO Avanzada (Herencia, Interfaces y Abstractas)

5.1 Herencia (extends) y @Override

Mecanismo polimórfico mediante el cual una clase secundaria asimila la estructura completa de un padre, pudiendo reescribir un comportamiento usando la anotación @Override.

```
// Clase base Padre
public class Unidad {
    public void mover() {
        System.out.println("La unidad avanza caminando.");
    }
}

// Clase hija heredera
public class Caballero extends Unidad {
    @Override
    public void mover() {
        System.out.println("El caballero avanza velozmente a caballo.");
    }
}
```

5.2 Clases Abstractas (abstract)

Estructuras conceptuales parciales imposibles de instanciar de forma aislada, que definen variables de estado concretas y unifican comportamientos genéticos rígidos.

```
public abstract class PersonajeAbstracto {
    protected int puntosVida;

    public PersonajeAbstracto(int vidaInicial) {
        this.puntosVida = vidaInicial;
    }

    // Método normal con código heredable gratuito
    public void recibirDanio(int cantidad) {
        puntosVida -= cantidad;
    }

    // Método abstracto obligatorio sin cuerpo
    public abstract void ejecutarHabilidadEspecial();
}
```

5.3 Interfaces (interface)

Contratos estrictos de comportamiento puro que inyectan habilidades operativas a clases sin la obligación de que compartan un tronco genético común.

```

public interface Atacante {
    // Firma de método obligatoria
    void lanzarAtaque(String objetivo);
}

public class Torreta implements Atacante {
    @Override
    public void lanzarAtaque(String objetivo) {
        System.out.println("Torreta dispara láser automático hacia: " + objetivo);
    }
}

```

Tabla Comparativa Avanzada

Característica	Clase Abstracta (abstract class)	Interfaz (interface)
Paradigma	Define la identidad del elemento (Qué es).	Define capacidades operativas (Qué sabe hacer).
Atributos	Soporta variables normales y privadas que mutan.	Solo variables públicas, estáticas y finales (constantes).
Mecanismo de uso	Herencia única mediante la instrucción extends.	Implementación múltiple mediante la instrucción implements.

Bloque 6: Comparación de Objetos

6.1 Método equals()

Inspecciona el contenido estructural interno de dos instancias para dictaminar igualdad semántica, eludiendo la comparación superficial de punteros de memoria del operador ==.

```

String cadenaUno = new String("Espada");
String cadenaDos = new String("Espada");

```

```
// Compara el contenido de los objetos de manera segura
if (cadenaUno.equals(cadenaDos)) {
    System.out.println("Las cadenas contienen el mismo texto.");
}
```

6.2 Método compareTo()

Indispensable para establecer un ordenamiento natural, devolviendo enteros negativos, positivos o cero en función de la jerarquía lexicográfica o numérica analizada.

```
String heroeA = "Aragorn";
String heroeB = "Gandalf";

int ordenacion = heroeA.compareTo(heroeB);
if (ordenacion < 0) {
    System.out.println("Aragorn se posiciona antes que Gandalf alfabéticamente.");
}
```

Bloque 7: Manipulación de Textos y Envoltorios

7.1 Clases Envoltorio (Wrappers)

Clases de objetos sofisticados diseñadas para encapsular y dotar de métodos complejos a los tipos de datos primitivos esenciales.

```
// Conversión de primitivo a Objeto Envoltorio mediante Autoboxing
Integer vidaContenedor = Integer.valueOf(100);
// Parseo de texto a tipo primitivo numérico directo
int vidaConvertida = Integer.parseInt("150");
System.out.println("Vida convertida: " + vidaConvertida);
```

7.2 Métodos trim() y toLowerCase()

Funciones de saneamiento diseñadas para purgar espacios en blanco residuales externos y estandarizar textos a minúsculas homogéneas.

```
String entradaSucia = " MagoOscuro ";
String entradaLimpia = entradaSucia.trim().toLowerCase();
System.out.println("Resultado: " + entradaLimpia + ""); // Imprime 'magooscuro'
```

7.3 Trocear textos con split()

Segmenta un String delimitado generando de forma automática un array ordenado con los fragmentos resultantes.

```
String datosMonstruo = "Orco-80-Fuerza";  
String[] partes = datosMonstruo.split("-");  
System.out.println("Raza: " + partes[0]);  
System.out.println("Vida base: " + partes[1]);
```

Bloque 8: Arrays y Colecciones Dinámicas

8.1 Arrays Estáticos

Estructuras de almacenamiento indexadas rígidas de tamaño fijo inmutable asignadas en memoria al inicializarse.

```
int[] nivelesEquipamiento = new int[3];  
nivelesEquipamiento[0] = 5;  
nivelesEquipamiento[1] = 12;  
System.out.println("Espacio asignado inicial: " + nivelesEquipamiento[0]);
```

8.2 Utilidades con java.util.Arrays

Proporciona algoritmos directos integrados para manipular, ordenar de forma creciente y mapear arrays nativos estáticos.

```
import java.util.Arrays;  
  
int[] puntuaciones = {90, 45, 78};  
Arrays.sort(puntuaciones); // Ordena el array de menor a mayor automáticamente  
System.out.println(Arrays.toString(puntuaciones)); // Imprime [45, 78, 90]
```

8.3 Listas Dinámicas (ArrayList)

Colección secuencial indexada de tamaño dinámico automatizado expandible de forma transparente según la demanda de inserción.

```
import java.util.ArrayList;  
  
ArrayList<String> bolsaObjetos = new ArrayList<>();  
bolsaObjetos.add("Poción");  
bolsaObjetos.add("Escudo");  
System.out.println("Elemento en el índice 0: " + bolsaObjetos.get(0));
```


8.4 Iteradores (Iterator)

Mecanismo de puntero secuencial seguro y optimizado diseñado para purgar elementos de colecciones dinámicas en medio de un bucle de lectura sin lanzar excepciones en tiempo de ejecución.

```
import java.util.ArrayList;
import java.util.Iterator;

ArrayList<String> inventario = new ArrayList<>();
inventario.add("Poción Usada");
inventario.add("Espada");

Iterator<String> cursor = inventario.iterator();
while (cursor.hasNext()) {
    if (cursor.next().equals("Poción Usada")) {
        cursor.remove(); // Eliminación segura del elemento actual sin romper el bucle
    }
}
```

8.5 Conjuntos (Set / HashSet)

Colecciones de datos no indexadas optimizadas estructuralmente para impedir y filtrar de raíz la adición de elementos duplicados.

```
import java.util.HashSet;

HashSet<Integer> codigosIdentificadores = new HashSet<>();
codigosIdentificadores.add(500);
codigosIdentificadores.add(500); // Intento duplicado, el conjunto lo ignorará por completo
System.out.println("Elementos únicos totales: " + codigosIdentificadores.size()); // Imprime 1
```

8.6 Mapas (Map / HashMap)

Estructuras de almacenamiento avanzadas organizadas mediante parejas inseparables de Clave Única y Valor Indexable asociados (similar a un diccionario de datos).

```
import java.util.HashMap;

HashMap<String, Integer> registroEdades = new HashMap<>();
registroEdades.put("Aragorn", 87);
registroEdades.put("Gandalf", 2000);
System.out.println("Edad vinculada a Aragorn: " + registroEdades.get("Aragorn"));
```

